МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

_____

(полное наименование учебного заведения)

Кафедра _____

КУРСОВОЙ ПРОЕКТ

по дисциплине «Базы данных»

на тему: «Проектирование и разработка базы данных “Кинотеатр”»

Выполнил: студент группы _____

_____

(фамилия, имя, отчество)

Проверил: Федоров И. А.

Оценка _____

_____ 2026

Содержание

Раздел	Стр.
Введение	4
1. Постановка задачи	6
1.1. Теоретические сведения по проектированию реляционных баз данных	6
1.2. Анализ предметной области	8
1.2.1. Обследование организации	8
1.2.2. Пользователи системы и их потребности	8
1.2.3. Перечень функций системы	9
1.2.4. Перечень задач и запросов, входные и выходные данные	9
1.2.5. Возможные изменения информационных потребностей	10
1.3. Модель «сущность — связь» для базы данных задачи	10
1.3.1. Выделенные наборы объектов	10
1.3.2. Атрибуты наборов объектов	10
1.3.3. Связи между наборами объектов	11
1.3.4. ER-диаграмма	12
1.4. Схема реляционной базы данных задачи	12
1.4.1. Правила перехода к схеме отношений	12
1.4.2. Описание таблиц	13
1.4.3. Связи между таблицами по конкретным полям	15
1.4.4. Вычисляемые столбцы и представления	16
1.4.5. Нормализация схемы	17
1.4.6. Правила целостности данных	18
1.4.7. Реализация структуры базы данных	18
1.4.8. Заполнение таблиц контрольными данными	19
2. Разработка программного приложения	20
2.1. Описание структуры и основных методов приложения	20
2.1.1. Выбор средств разработки	20
2.1.2. Структура приложения	20
2.1.3. Реализованные функции	22
2.1.4. Пример реализации: продажа билета	22
2.2. Результаты тестирования и инструкция по использованию	23
2.2.1. Результаты тестирования	23

Раздел	Стр.
2.2.2. Примеры экранных форм	24
2.2.3. Инструкция по использованию системы	26
Заключение	27
Список использованной литературы	29
Приложение 1. Исходный текст программного приложения	30

Введение

Современный кинотеатр ежедневно обрабатывает большой объём информации: расписание сеансов, сведения о фильмах и залах, продажу и возврат билетов, учёт постоянных клиентов и работу кассиров. Пока эти сведения ведутся на бумаге или в разрозненных таблицах, возникают типичные проблемы: одно и то же место продаётся дважды, расписание составляется с накладками, выручку за день приходится считать вручную, а данные о фильме повторяются в каждой записи о сеансе и при изменении названия или длительности правятся в десятках мест.

Актуальность работы состоит в том, что перечисленные проблемы решаются переходом к базе данных: сведения хранятся однократно, связи между ними задаются средствами системы управления базами данных, а правила целостности не позволяют внести противоречивые данные.

Цель курсового проекта — спроектировать и разработать базу данных «Кинотеатр» и программное приложение для работы с ней.

Для достижения цели решались следующие **задачи**:

1. исследовать предметную область — работу кинотеатра — и определить, какие сведения должна хранить система;
2. составить перечень функций и запросов, которые должна выполнять база данных, определить входные и выходные данные;
3. выделить сущности, их атрибуты и ключи, установить связи между сущностями и построить инфологическую модель;
4. преобразовать инфологическую модель в схему реляционной базы данных, разделив связи «многие ко многим»;
5. провести нормализацию отношений и задать правила целостности данных;
6. реализовать структуру базы данных средствами выбранной системы управления базами данных и заполнить таблицы контрольными данными;
7. разработать и отладить программное приложение для работы с базой данных;

8. оформить пояснительную записку.

Объект исследования — деятельность кинотеатра по организации показа фильмов и продаже билетов. **Предмет исследования** — структура данных, необходимая для автоматизации этой деятельности.

Записка состоит из двух разделов: в первом приведены постановка задачи, теоретические сведения, анализ предметной области, инфологическая модель и схема реляционной базы данных; во втором описано разработанное приложение, результаты тестирования, экранные формы и инструкция пользователя. В приложении помещён исходный текст программы.

1. Постановка задачи

Требуется разработать информационную систему кинотеатра, которая обеспечивает хранение сведений о фильмах, залах, расписании сеансов, клиентах, сотрудниках и проданных билетах, а также выполнение операций добавления, удаления, изменения и поиска данных с отбором информации по заданным признакам.

Требования к разрабатываемой базе данных и приложению:

- количество таблиц — не менее четырёх;
- таблицы должны быть связаны между собой;
- в структуре должны присутствовать вычисляемые столбцы;
- объём каждой таблицы — не менее 10 записей;
- приложение должно выполнять добавление, удаление, поиск и отбор информации.

1.1. Теоретические сведения по проектированию реляционных баз данных

База данных — это совокупность данных, организованных по определённым правилам и предназначенных для совместного использования. **Система управления базами данных (СУБД)** — программное обеспечение, которое создаёт базу данных, обеспечивает доступ к данным, их изменение и защиту.

В основе **реляционной модели**, предложенной Э. Коддом, лежит понятие **отношения** — таблицы, строки которой называются кортежами (записями), а столбцы — атрибутами (полями). Каждый атрибут принимает значения из некоторого **домена** — множества допустимых значений одного типа.

Проектирование базы данных выполняется в три этапа: **инфологическое (концептуальное) проектирование** — описание предметной области в терминах «сущность — связь» без привязки к конкретной СУБД, результатом которого является ER-диаграмма; **даталогическое (логическое) проектирование** — переход к модели данных конкретного типа СУБД, когда сущности превращаются в таблицы, атрибуты в поля, а связи во внешние ключи; **физическое проектирование** — описание размещения данных в памяти: типы данных, индексы, файлы базы данных.

Основные понятия: **сущность** — объект предметной области, сведения о котором хранятся в базе данных; **набор объектов** — множество однотипных сущностей с одинаковым набором атрибутов; **первичный ключ (PK)** — атрибут или группа атрибутов, однозначно определяющих запись; **внешний ключ (FK)** — атрибут, значения которого ссылаются на первичный ключ другой таблицы (так реализуются связи); **связь** — ассоциация между наборами объектов, которая бывает «один к одному» (1:1), «один ко многим» (1:M) и «многие ко многим» (M:M).

Связь M:M в реляционной базе данных напрямую реализовать нельзя, поэтому при переходе от ER-модели к схеме отношений такая связь разделяется на две связи 1:M с помощью дополнительной (связующей) таблицы, первичный ключ которой составлен из внешних ключей обеих исходных таблиц.

Чтобы избежать **аномалий модификации данных** — дублирования сведений и противоречий, возникающих при добавлении, изменении и удалении записей, — проводится **нормализация** отношений:

- **Первая нормальная форма (1НФ)**: все поля таблицы содержат только одно (атомарное) значение, повторяющихся групп нет.
- **Вторая нормальная форма (2НФ)**: отношение находится в 1НФ, и каждое неключевое поле функционально полно зависит от первичного ключа, то есть не зависит от части составного ключа.
- **Третья нормальная форма (3НФ)**: отношение находится во 2НФ, и неключевые поля не зависят друг от друга (отсутствуют транзитивные зависимости).

Наряду с нормализацией задаются **правила целостности данных**: целостность сущностей (первичный ключ не может быть пустым и не повторяется), ссылочная целостность (значение внешнего ключа либо пустое, либо совпадает со значением первичного ключа связанной таблицы) и целостность, определяемая предметной областью (ограничения на значения полей).

1.2. Анализ предметной области

1.2.1. Обследование организации

Кинотеатр располагает несколькими **залами**, которые отличаются числом рядов, количеством мест в ряду и типом оборудования (2D, 3D, IMAX, VIP-зал). Вместимость зала определяется произведением числа рядов на число мест в ряду.

В прокате находятся **фильмы**. О каждом фильме хранятся название, режиссёр, страна и год выпуска, длительность в минутах и возрастной рейтинг. Фильм относится к одному или нескольким **жанрам**, и, наоборот, к одному жанру относится множество фильмов.

Администратор составляет расписание — набор **сеансов**. Сеанс характеризуется датой, временем начала, залом, фильмом, форматом показа и базовой ценой билета. В одном зале в одно и то же время не может идти два сеанса.

Клиенты покупают **билеты**. Часть посетителей регистрируется в системе лояльности: о них хранятся фамилия, имя, телефон, электронная почта, дата регистрации и размер персональной скидки. Билет продаётся на конкретный сеанс, на конкретные ряд и место; одно и то же место на один сеанс не может быть продано дважды. В билете фиксируются дата продажи, цена без скидки, размер применённой скидки и статус билета (продан, забронирован, возвращён). Итоговая цена билета вычисляется автоматически.

Билеты оформляют **сотрудники** кинотеатра — кассиры и администраторы. О сотруднике хранятся фамилия, имя, должность, телефон и дата приёма на работу. По каждому проданному билету известно, кто из сотрудников его оформил, что позволяет вести учёт работы кассиров.

1.2.2. Пользователи системы и их потребности

Пользователь	Задачи, решаемые с помощью системы
Кассир	продажа и возврат билетов, поиск свободных мест на сеанс, поиск клиента
Администратор	составление расписания, ведение справочников фильмов, жанров и залов

Пользователь	Задачи, решаемые с помощью системы
Руководитель кинотеатра	отчёты о выручке, загрузке залов, популярности фильмов и работе кассиров

1.2.3. Перечень функций системы

Информационная система должна обеспечивать:

1. ведение справочников: добавление, изменение и удаление записей о фильмах, жанрах, залах, клиентах и сотрудниках;
2. составление расписания сеансов с контролем занятости залов;
3. продажу билетов с автоматическим расчётом цены с учётом скидки клиента;
4. контроль занятости мест: запрет повторной продажи одного места на один сеанс;
5. возврат билета;
6. поиск и отбор информации по заданным признакам;
7. формирование отчётов о работе кинотеатра.

1.2.4. Перечень задач и запросов, входные и выходные данные

№	Запрос к базе данных	Входные данные	Выходные данные
Q1	Расписание сеансов на заданную дату	дата	время, фильм, зал, формат, цена
Q2	Свободные места на выбранный сеанс	код сеанса	список рядов и мест
Q3	Фильмы заданного жанра	название жанра	список фильмов
Q4	Билеты, купленные клиентом	код клиента	дата, фильм, ряд, место, цена
Q5	Выручка кинотеатра за период	даты начала и конца	сумма выручки, число билетов
Q6	Заполненность залов по сеансам	дата	продано, вместимость, процент
Q7	Рейтинг фильмов по проданным билетам	период	фильмы по числу билетов
Q8	Количество билетов по кассирам	период	сотрудник, число билетов, сумма

1.2.5. Возможные изменения информационных потребностей

При развитии кинотеатра возможны следующие изменения, которые учитывались при проектировании: введение ценовых категорий мест в зале; продажа билетов через сайт и мобильное приложение; система абонементов и бонусных баллов; учёт нескольких кинотеатров сети. Структура базы данных спроектирована так, чтобы эти изменения сводились к добавлению новых таблиц и полей, не затрагивая существующие связи.

1.3. Модель «сущность — связь» для базы данных задачи

Построение ER-диаграммы выполнено по методике: в описании предметной области выделены существительные, обозначающие будущие сущности, для них определены атрибуты и ключевые атрибуты, после чего установлены связи.

1.3.1. Выделенные наборы объектов

№	Набор объектов	Описание	Ключ
1	ЖАНР	справочник жанров фильмов	Код жанра
2	ФИЛЬМ	фильм, находящийся в прокате	Код фильма
3	ЗАЛ	зал кинотеатра	Код зала
4	СЕАНС	показ фильма в зале в заданное время	Код сеанса
5	КЛИЕНТ	зарегистрированный покупатель	Код клиента
6	СОТРУДНИК	работник кинотеатра	Код сотрудника
7	БИЛЕТ	билет на место конкретного сеанса	Код билета

1.3.2. Атрибуты наборов объектов

Ключевые атрибуты выделены полужирным шрифтом.

ЖАНР: **Код жанра**, Название жанра.

ФИЛЬМ: **Код фильма**, Название, Режиссёр, Страна, Год выпуска, Длительность (мин), Возрастной рейтинг.

ЗАЛ: **Код зала**, Название зала, Количество рядов, Мест в ряду, Тип зала, *Всего мест* (вычисляемый атрибут).

СЕАНС: **Код сеанса**, Код фильма, Код зала, Дата сеанса, Время начала, Базовая цена билета, Формат показа.

КЛИЕНТ: **Код клиента**, Фамилия, Имя, Телефон, Электронная почта, Дата регистрации, Размер скидки (%).

СОТРУДНИК: **Код сотрудника**, Фамилия, Имя, Должность, Телефон, Дата приёма на работу.

БИЛЕТ: **Код билета**, Код сеанса, Код клиента, Код сотрудника, Ряд, Место, Дата продажи, Цена без скидки, Размер скидки, Статус, *Итоговая цена* (вычисляемый атрибут).

1.3.3. Связи между наборами объектов

№	Связь	Наборы объектов	Тип
1	относится к	ФИЛЬМ — ЖАНР	M:M
2	демонстрируется	ФИЛЬМ — СЕАНС	1:M
3	проводится в	ЗАЛ — СЕАНС	1:M
4	продаётся на	СЕАНС — БИЛЕТ	1:M
5	покупает	КЛИЕНТ — БИЛЕТ	1:M
6	оформляет	СОТРУДНИК — БИЛЕТ	1:M

Пояснения к связям:

- Один фильм демонстрируется на многих сеансах, но каждый сеанс относится к одному фильму — связь 1:M. Аналогично устроены связи 3–6.
- Связь «ФИЛЬМ — ЖАНР» является связью «многие ко многим»: фильм может относиться к нескольким жанрам, а каждый жанр объединяет множество фильмов. Согласно методике такая связь разделяется на две связи 1:M с помощью дополнительной сущности **ЖАНР ФИЛЬМА**, которая содержит только коды фильма и жанра.
- Связь «КЛИЕНТ — БИЛЕТ» необязательна со стороны клиента: билет может быть продан незарегистрированному посетителю, в этом случае код клиента в билете не указывается.

1.3.4. ER-диаграмма

После разделения связи «многие ко многим» получается уточнённая ER-диаграмма, в которой присутствуют только связи «один ко многим». Такая диаграмма является прототипом будущей базы данных: сущности становятся таблицами, а атрибуты — полями.

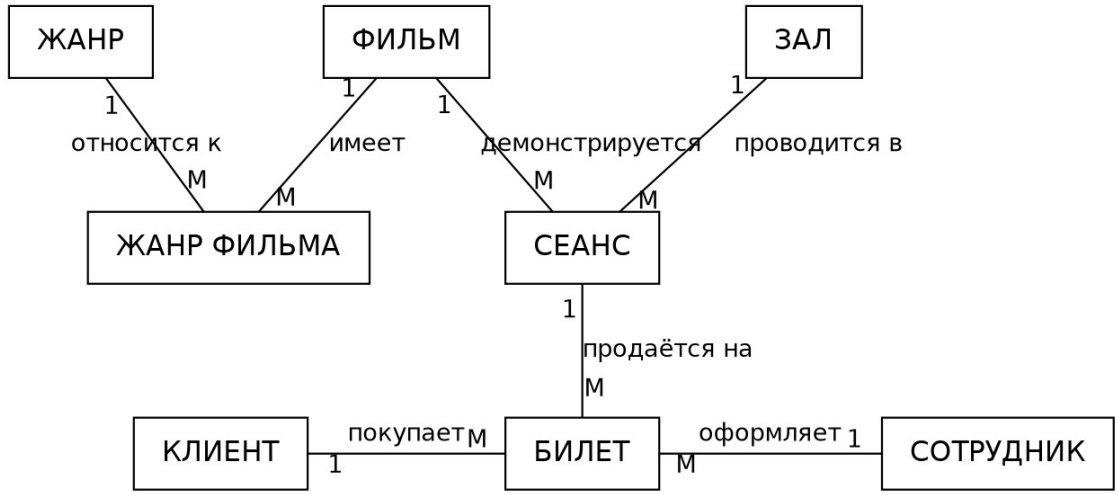


Рисунок 1 — Инфологическая модель базы данных «Кинотеатр»

1.4. Схема реляционной базы данных задачи

1.4.1. Правила перехода к схеме отношений

При преобразовании ER-модели в схему реляционной базы данных применены правила:

1. каждый набор объектов становится таблицей, сущность — строкой, атрибут — столбцом;
2. ключ набора объектов становится первичным ключом таблицы;
3. связь 1:M реализуется переносом первичного ключа таблицы со стороны «один» в таблицу со стороны «многие» в качестве внешнего ключа;
4. связь M:M реализуется через связующую таблицу с составным первичным ключом.

В результате получено **восемь таблиц**, одна из которых (FILM_ZHANR) является связующей.

1.4.2. Описание таблиц

Обозначения: PK — первичный ключ, FK — внешний ключ.

Структура таблиц приведена в таблице 1.

Таблица 1 — Структура таблиц базы данных «Кинотеатр»

Таблица	Столбец	Тип	Ключ	Описание
ZHANR	id_zhanra	INTEGER	PK	код жанра
	nazvanie	TEXT		название жанра, уникальное
FILM	id_filma	INTEGER	PK	код фильма
	nazvanie	TEXT		название фильма
	rezhisser	TEXT		режиссёр
	strana	TEXT		страна производства
	god_vypuska	INTEGER		год выпуска
	dlitelnost_min	INTEGER		длительность в минутах
	vozrastnoy_reyting	TEXT		0+, 6+, 12+, 16+, 18+
FILM_ZHANR	id_filma	INTEGER	PK, FK	фильм
	id_zhanra	INTEGER	PK, FK	жанр
ZAL	id_zala	INTEGER	PK	код зала
	nazvanie	TEXT		название зала
	kolichestvo_ryadov	INTEGER		количество рядов
	mest_v_ryadu	INTEGER		мест в ряду
	tip_zala	TEXT		2D, 3D, IMAX, VIP

Таблица	Столбец	Тип	Ключ	Описание
	vsego_mest	INTEGER		вычисляемый: рядов × мест в ряду
SEANS	id_seansa	INTEGER	PK	код сеанса
	id_filma	INTEGER	FK	фильм
	id_zala	INTEGER	FK	зал
	data_seansa	TEXT		дата сеанса
	vremya_nachala	TEXT		время начала
	bazovaya_cena	REAL		базовая цена билета
	format_pokaza	TEXT		2D, 3D, IMAX
KLIENT	id_klienta	INTEGER	PK	код клиента
	familiya, imya	TEXT		фамилия и имя
	telefon	TEXT		телефон, уникальный
	email	TEXT		электронная почта
	data_registracii	TEXT		дата регистрации
	skidka_procent	REAL		размер скидки, %
SOTRUDNIK	id_sotrudnika	INTEGER	PK	код сотрудника
	familiya, imya	TEXT		фамилия и имя
	dolzhnost	TEXT		должность
	telefon	TEXT		телефон
	data_priema	TEXT		дата приёма на работу
BILET	id_bileta	INTEGER	PK	код билета
	id_seansa	INTEGER	FK	сеанс

Таблица	Столбец	Тип	Ключ	Описание
	id_klienta	INTEGER	FK	клиент (может отсутствовать)
	id_sotrudnika	INTEGER	FK	сотрудник, оформивший билет
	ryad, mesto	INTEGER		ряд и место
	data_prodazhi	TEXT		дата продажи
	cena_bez_skidki	REAL		цена без скидки
	skidka_procent	REAL		применённая скидка, %
	status	TEXT		продан, забронирован, возвращён
	itogovaya_cena	REAL		вычисляемый: цена со скидкой

1.4.3. Связи между таблицами по конкретным полям

№	Таблица и поле (FK)	Ссылается на (PK)	Связь
1	FILM_ZHANR.id_filma	FILM.id_filma	M:M (1-я часть)
2	FILM_ZHANR.id_zhanra	ZHANR.id_zhanra	M:M (2-я часть)
3	SEANS.id_filma	FILM.id_filma	1:M
4	SEANS.id_zala	ZAL.id_zala	1:M
5	BILET.id_seansa	SEANS.id_seansa	1:M
6	BILET.id_klienta	KLIENT.id_klienta	1:M
7	BILET.id_sotrudnika	SOTRUDNIK.id_sotrudnika	1:M

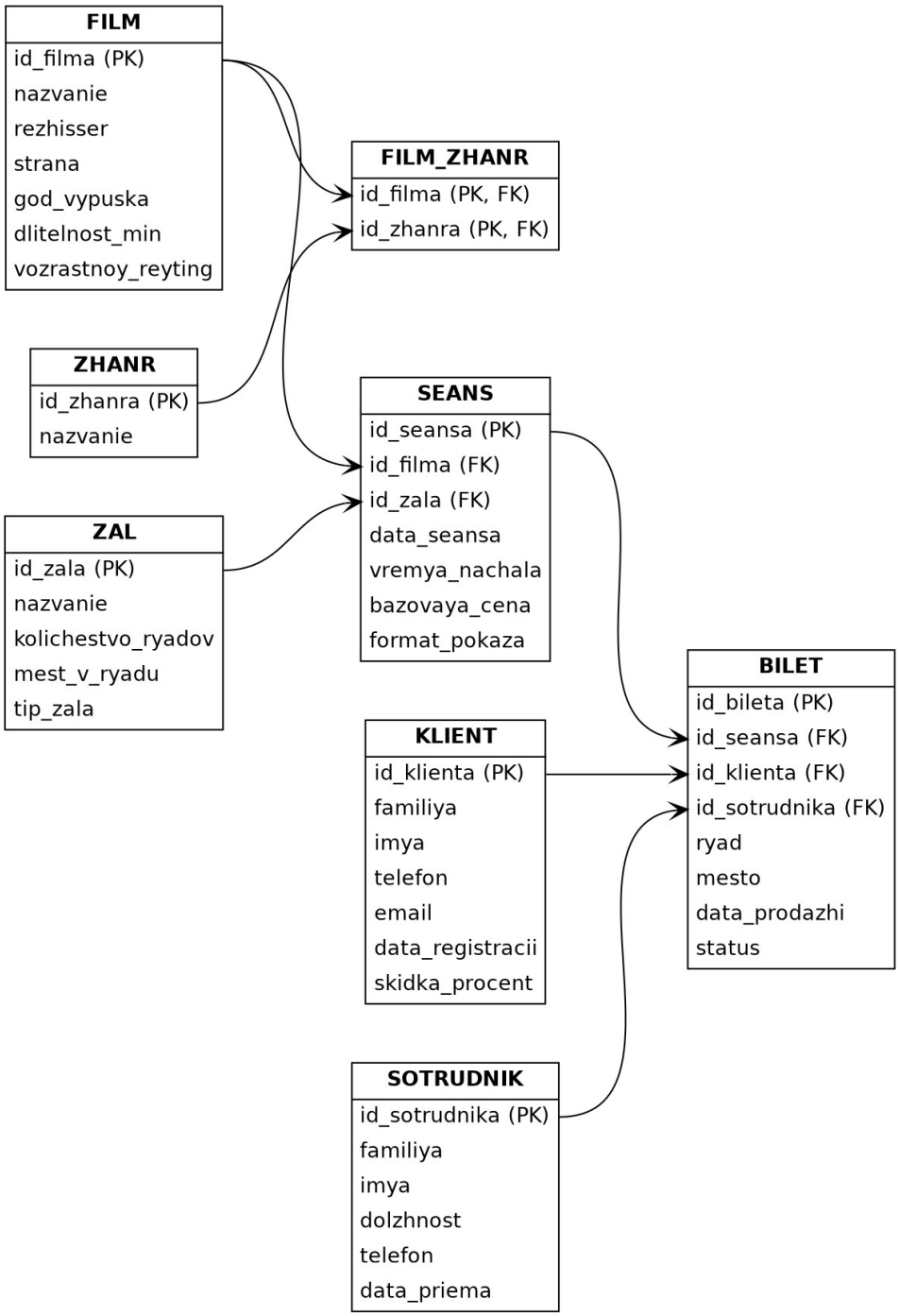


Рисунок 2 — Схема реляционной базы данных «Кинотеатр»

1.4.4. Вычисляемые столбцы и представления

В структуре базы данных предусмотрены вычисляемые столбцы, значения которых не вводятся пользователем, а рассчитываются СУБД:

- **ZAL.vsego_mest** — вместимость зала, равная произведению числа рядов на число мест в ряду;
- **BILET.itogovaya_cena** — итоговая стоимость билета, равная цене без скидки, умноженной на (1 − скидка / 100).

Для отчётности созданы представления (виртуальные таблицы): **v_raspisanie** — расписание сеансов с названиями фильма и зала; **v_seans_svodka** — количество проданных билетов, выручка и процент заполненности зала по каждому сеансу.

1.4.5. Нормализация схемы

1НФ. Все поля таблиц атомарны, повторяющихся групп нет. Множественные сведения вынесены в отдельные таблицы: жанры фильма — в FILM_ZHANR, сеансы — в SEANS, билеты — в BILET. Фамилия и имя хранятся в отдельных полях.

2НФ. Все неключевые поля зависят от полного первичного ключа. Проверка существенна для связующей таблицы FILM_ZHANR: она состоит только из ключевых полей, поэтому частичных зависимостей в ней нет. Название жанра в эту таблицу не включено, так как зависело бы только от id_zhanra.

3НФ. Транзитивные зависимости отсутствуют: в таблице SEANS не хранятся название и длительность фильма — они зависели бы от id_filma и находятся в таблице FILM; в таблице SEANS не хранится вместимость зала — она вычисляется в таблице ZAL; в таблице BILET не дублируются фамилия клиента и фамилия кассира, хранятся только их коды.

Отдельно следует пояснить поля **cena_bez_skidki** и **skidka_procent** в таблице BILET. На первый взгляд они дублируют базовую цену сеанса и скидку клиента, однако это не нарушение нормальных форм: цена и скидка фиксируются в момент продажи и не должны изменяться задним числом при изменении цены сеанса или размера скидки клиента. Такое решение отражает требование предметной области.

Все восемь таблиц находятся в третьей нормальной форме.

1.4.6. Правила целостности данных

Правило предметной области	Реализация в базе данных
Каждая запись однозначно определяется	первичный ключ в каждой таблице
Сеанс ссылается на существующие фильм и зал	внешние ключи SEANS.id_filma, SEANS.id_zala
В одном зале в одно время только один сеанс	UNIQUE (id_zala, data_seansa, vremya_nachala)
Одно место на сеанс продаётся один раз	UNIQUE (id_seansa, ryad, mesto)
Жанр не повторяется у фильма дважды	составной первичный ключ FILM_ZHANR
Длительность фильма и цена положительны	CHECK (dlitelnost_min > 0), CHECK (bazovaya_cena > 0)
Скидка не превышает 50 %	CHECK (skidka_procent BETWEEN 0 AND 50)
Возрастной рейтинг из перечня	CHECK (vozrastnoy_reyting IN (‘0+’,‘6+’,‘12+’,‘16+’,‘18+’))
Статус билета из перечня	CHECK (status IN (‘продан’,‘забронирован’,‘возвращён’))
Нельзя удалить фильм, на который есть сеансы	ON DELETE RESTRICT

1.4.7. Реализация структуры базы данных

В качестве СУБД выбрана SQLite: база данных хранится в одном файле `kinoteatr.db` рядом с программой, не требует установки сервера и настройки, что удобно для настольного приложения.

Скрипт создания таблиц `kino_schema.sql` содержит описания всех восьми таблиц с первичными и внешними ключами, ограничениями CHECK и UNIQUE, вычисляемыми столбцами (`GENERATED ALWAYS AS`), индексами по полям связей и двумя представлениями.

1.4.8. Заполнение таблиц контрольными данными

Таблицы заполнены скриптом `kino_dannye.sql`. Объём данных подобран так, чтобы выполнялось требование «не менее 10 записей в каждой таблице» и чтобы на этих данных можно было проверить все функции приложения.

Таблица	Содержание	Записей
ZHANR	жанры фильмов	10
FILM	фильмы в прокате	12
FILM_ZHANR	жанры фильмов (связующая таблица)	24
ZAL	залы кинотеатра	10
SEANS	сеансы расписания	20
KLIENT	зарегистрированные клиенты	12
SOTRUDNIK	сотрудники кинотеатра	10
BILET	проданные билеты	25

Фрагмент таблицы ZAL (последний столбец вычисляется базой данных):

id_zala	nazvanie	kolichestvo_ryadov	mest_v_ryadu	tip_zala	vsego_mest
1	Красный	12	14	3D	168
2	Синий	10	12	2D	120
4	Большой	16	20	IMAX	320
6	Золотой	5	6	VIP	30

Так же автоматически рассчитывается итоговая цена билета: например, для билета № 1 ценой 650 руб. при скидке клиента 10 % в поле `itogovaya_cena` хранится значение 585 руб., а для билета № 13 ценой 450 руб. со скидкой 25 % — 337,50 руб.

2. Разработка программного приложения

2.1. Описание структуры и основных методов приложения

2.1.1. Выбор средств разработки

Приложение написано на языке **Python** с использованием библиотеки **tkinter** для построения оконного интерфейса и встроенного модуля **sqlite3** для работы с базой данных. Выбор сделан по следующим причинам:

- tkinter и sqlite3 входят в стандартную поставку Python, поэтому для запуска программы не нужно устанавливать дополнительные библиотеки;
- база данных хранится в одном файле рядом с программой, сервер базы данных не требуется;
- программа работает в операционной системе Windows и запускается одной командой.

Состав файлов проекта:

Файл	Назначение
`kinoteatr.py`	программа, 867 строк с комментариями
`kino_schema.sql`	создание таблиц, ключей, ограничений, индексов и представлений
`kino_dannye.sql`	заполнение таблиц контрольными данными
`kinoteatr.db`	файл базы данных

Если файла базы данных рядом с программой нет, при первом запуске она создаётся автоматически из двух скриптов.

2.1.2. Структура приложения

Программа построена по модульному принципу: работа с базой данных отделена от интерфейса. Приложение состоит из пяти классов.

Класс BazaDannyh отвечает за подключение к базе данных и выполнение запросов:

Основные методы: `__init__(fayl)` — подключается к файлу базы и включает контроль внешних ключей; `sozdat_bazu()` — создаёт таблицы и заполняет их

данными, если файла базы нет; `vybrat(sql, parametry)` — выполняет запрос на выборку и возвращает список записей; `vypolnit(sql, parametry)` — выполняет запрос на изменение данных; `zakryt()` — закрывает соединение.

Все запросы выполняются с параметрами: вместо значений в текст запроса подставляются знаки вопроса, а сами значения передаются отдельно. Это исключает ошибки при вводе кавычек и защищает от подстановки постороннего кода.

Класс VkladkaSpravochnik — универсальная вкладка для работы со справочниками. Один класс используется для пяти таблиц (фильмы, залы, клиенты, сотрудники, жанры): при создании ему передаются имя таблицы, список полей и их подписи на русском языке, поэтому один и тот же код не пришлось переписывать пять раз.

Основные методы: `obnovit()` — заполняет таблицу на экране с учётом строки поиска; `vybrat_zapis()` — переносит выбранную строку в поля ввода; `dobavit()`, `izmenit()`, `udalit()` — выполняют запросы INSERT, UPDATE и DELETE; `ochistit()` — очищает поля ввода.

Класс VkladkaSeansy — работа с расписанием. Фильм и зал выбираются из выпадающих списков, поэтому ввести несуществующий код нельзя. Методы добавления, изменения и удаления обрабатывают ошибки целостности: при попытке назначить два сеанса в одном зале на одно время выводится понятное сообщение, при попытке удалить сеанс с проданными билетами — предупреждение.

Класс VkladkaBilety — продажа и возврат билетов:

Основные методы: `zagruzit_spiski()` — загружает списки сеансов, клиентов и кассиров; `vybran_seans()` — ограничивает номера ряда и места размерами выбранного зала; `poschitat_cenu()` — рассчитывает цену билета с учётом скидки; `prodat()` — продаёт билет с проверкой занятости места; `pokazat_svobodnye()` — показывает свободные и занятые места; `vernut()` — оформляет возврат билета.

Класс VkladkaOtchety — построение отчётов. Метод `podstavit_parametry()` автоматически подставляет в поля подходящие значения (например, границы

периода продаж), метод `postroit()` выполняет запрос выбранного отчёта и выводит результат на экран.

Класс GlavnoeOkno — главное окно программы: создаёт подключение к базе данных, восемь вкладок и корректно закрывает соединение при выходе.

2.1.3. Реализованные функции

1. **Ведение справочников** — добавление, изменение и удаление фильмов, залов, клиентов, сотрудников и жанров.
2. **Поиск и отбор информации** — поиск по справочникам сразу по нескольким полям, отбор расписания по дате, отбор фильмов по жанру.
3. **Составление расписания** — добавление сеансов с контролем занятости залов.
4. **Продажа билетов** — автоматический расчёт цены с учётом скидки клиента, контроль занятости мест, просмотр свободных мест.
5. **Возврат билетов** — изменение статуса билета.
6. **Отчёты** — шесть отчётов о работе кинотеатра.

2.1.4. Пример реализации: продажа билета

Продажа билета — наиболее показательная операция, в которой участвуют несколько таблиц и проверок. Метод `prodat()` проверяет, что выбраны сеанс и кассир, что номера ряда и места не выходят за размеры зала, определяет размер скидки клиента (для покупателя без карты она равна нулю) и выполняет вставку записи в таблицу BILET. Если место уже продано, база данных отклоняет вставку по ограничению UNIQUE, и программа показывает понятное сообщение; при успешной продаже выводится итоговая стоимость, рассчитанная вычисляемым столбцом.

```
try:
    self.bd.vypolnit(
        'INSERT INTO bilet (id_seansa, id_klienta, id_sotrudnika, ryad, mesto, '
        'data_prodazhi, cena_bez_skidki, skidka_procent, status) '
        "VALUES (?, ?, ?, ?, ?, date('now'), ?, ?, 'продан')",
        (seans['id_seansa'], kod_klienta, kod_sotrudnika, ryad, mesto,
         seans['bazovaya_cena'], skidka))
except sqlite3.IntegrityError:
    messagebox.showerror('Ошибка',
                         f'Место {mesto} в ряду {ryad} на этот сеанс уже продано.')
    return
```

Таким образом, правила, заданные при проектировании базы данных, продолжают работать и из приложения: программа не обходит ограничения, а сообщает пользователю о них понятным языком.

2.2. Результаты тестирования и инструкция по использованию

2.2.1. Результаты тестирования

Тестирование проводилось на заполненной базе данных. Проверялась как обычная работа, так и попытки ввести неверные данные.

№	Что проверялось	Результат
1	Продажа билета клиенту со скидкой	Билет продан, цена рассчитана верно
2	Повторная продажа того же места	Отклонено: «Место 9 в ряду 9 на этот сеанс уже продано»
3	Продажа билета в несуществующий ряд (99)	Отклонено: «В зале “Синий” 10 рядов по 12 мест»
4	Просмотр свободных мест	Показано: всего 120 мест, свободно 117
5	Возврат билета	Статус билета изменён на «возвращён»
6	Поиск по справочнику фильмов по слову «Нолан»	Найдено 3 записи
7	Построение шести отчётов	Все отчёты построены, данные совпадают с таблицами
8	Добавление и удаление записи в справочнике	Записей стало 11, после удаления снова 10
9	Попытка удалить зал, на который назначены сеансы	Отклонено: «Запись удалить нельзя: на неё ссылаются записи других таблиц»

Отдельно проверена работа вычисляемых столбцов: для зала «Большой» (16 рядов по 20 мест) значение `vsego_mest` равно 320; для билета ценой 650 руб. со скидкой 10 % значение `itogovaya_cena` равно 585 руб.

Ошибок в работе приложения не выявлено. Все ограничения целостности, заданные при проектировании, срабатывают, а программа выводит понятные сообщения вместо системных ошибок.

2.2.2. Примеры экранных форм

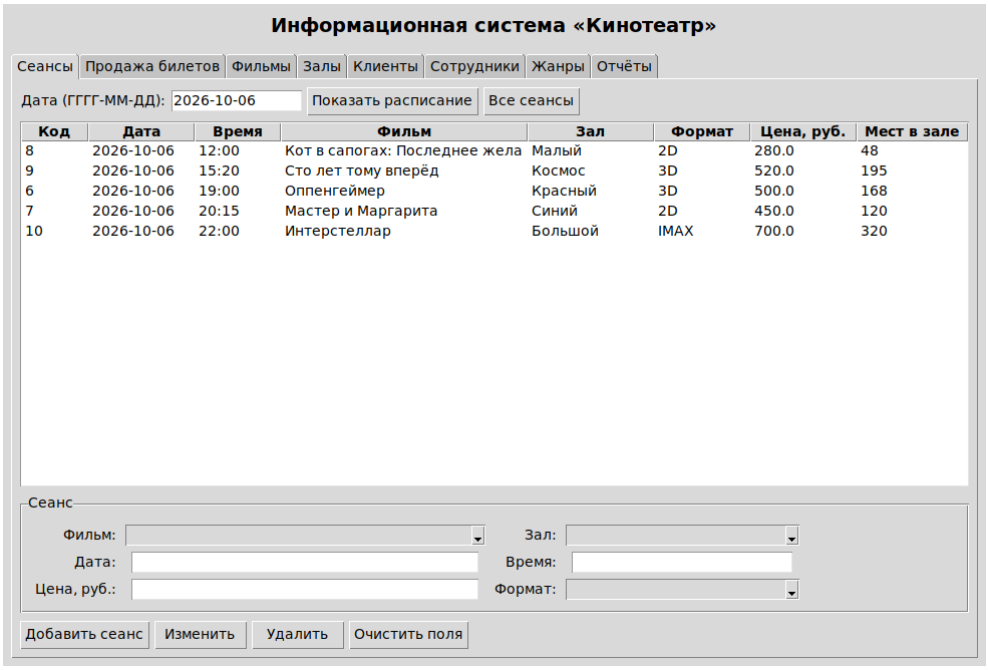


Рисунок 3 — Вкладка «Сеансы»: расписание на выбранную дату

На вкладке «Сеансы» отображается расписание. В поле даты можно указать день и получить сеансы только за этот день. При добавлении сеанса фильм и зал выбираются из выпадающих списков.

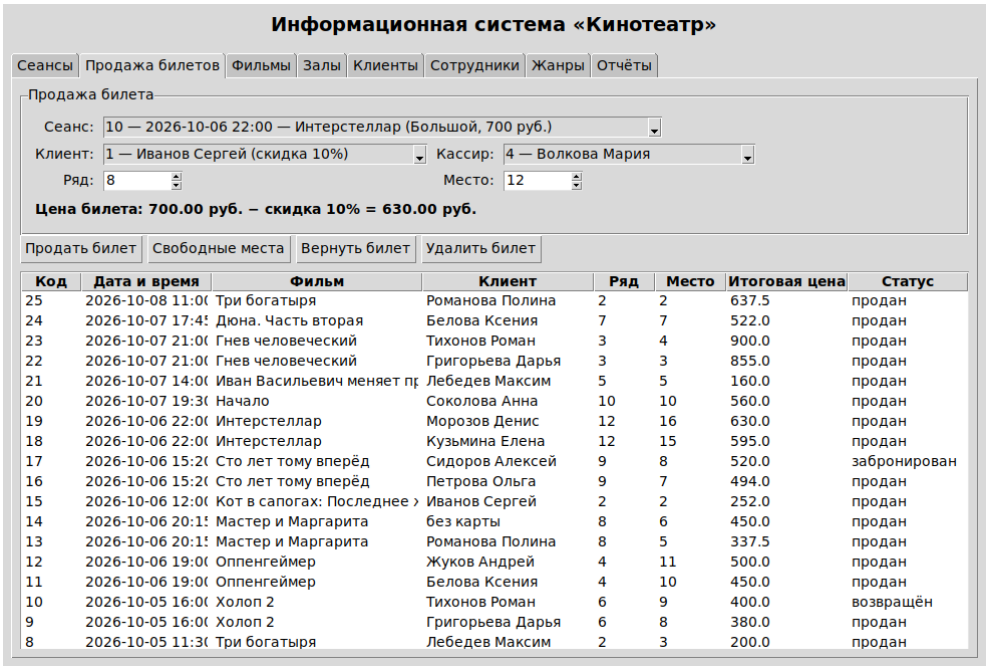


Рисунок 4 — Вкладка «Продажа билетов»

При выборе сеанса и клиента программа сама рассчитывает цену: базовая цена сеанса 700 руб., скидка клиента 10 %, к оплате 630 руб. Номера ряда и места ограничены размерами выбранного зала.

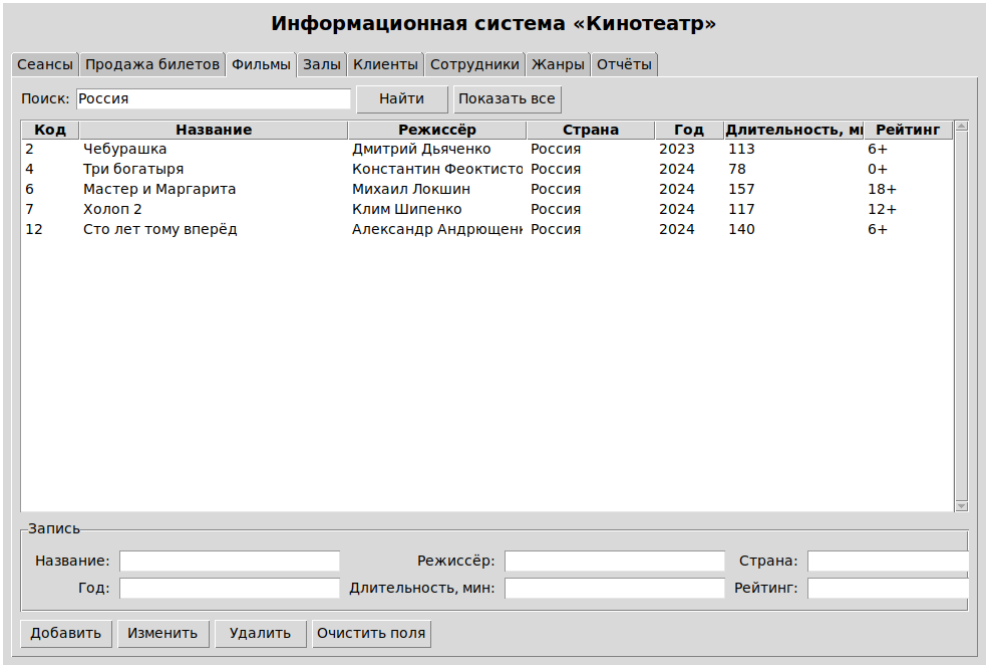


Рисунок 5 — Справочник фильмов с поиском по слову «Россия»

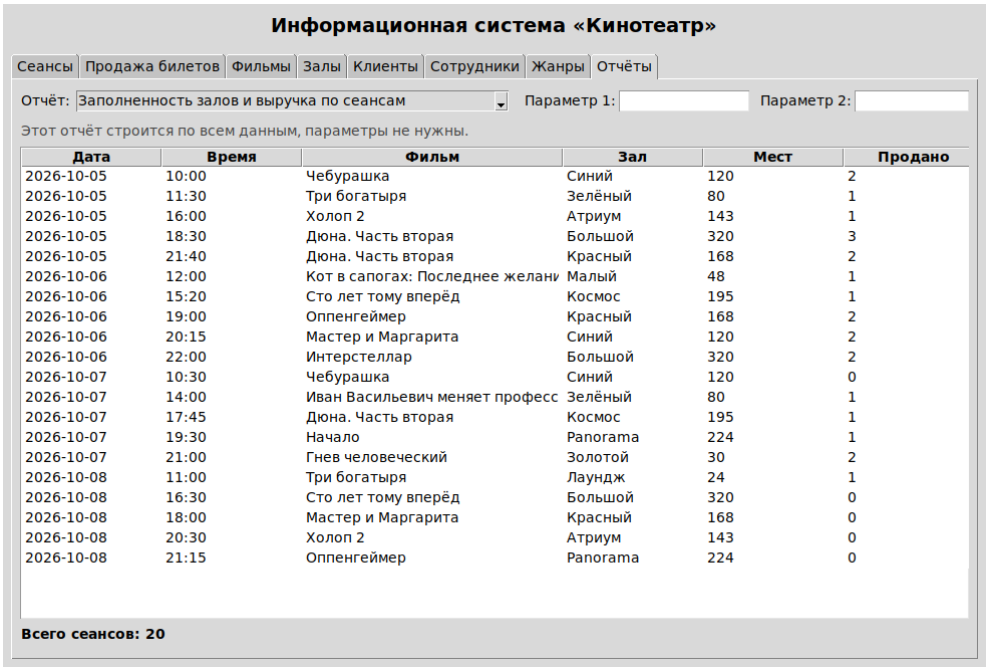


Рисунок 6 — Отчёт о заполненности залов и выручке по сеансам

2.2.3. Инструкция по использованию системы

1. **Запуск.** Поместить файлы `kinoteatr.py`, `kino_schema.sql`, `kino_dannye.sql` и `kinoteatr.db` в одну папку и выполнить команду `python kinoteatr.py`. Требуется Python версии 3.8 или новее; дополнительные библиотеки устанавливать не нужно.
2. **Расписание.** Вкладка «Сеансы»: ввести дату и нажать «Показать расписание»; кнопка «Все сеансы» снимает отбор. Чтобы добавить сеанс, выбрать фильм и зал, ввести дату, время, цену и формат, нажать «Добавить сеанс». Чтобы изменить сеанс, выбрать его в таблице, исправить поля и нажать «Изменить».
3. **Продажа билета.** Вкладка «Продажа билетов»: выбрать сеанс, клиента (или «без карты»), кассира, указать ряд и место, нажать «Продать билет». Кнопка «Свободные места» показывает, какие места заняты и сколько свободно.
4. **Возврат билета.** Выбрать билет в таблице и нажать «Вернуть билет»: статус билета изменится на «возвращён», место снова станет свободным.
5. **Справочники.** Вкладки «Фильмы», «Залы», «Клиенты», «Сотрудники», «Жанры»: выбрать запись в таблице — её значения появятся в полях ввода; кнопки «Добавить», «Изменить», «Удалить» выполняют соответствующие действия. Строка поиска отбирает записи сразу по нескольким полям.
6. **Отчёты.** Вкладка «Отчёты»: выбрать отчёт из списка — параметры подставятся автоматически; при необходимости изменить их и нажать «Построить отчёт».

Заключение

В ходе курсового проекта спроектирована и разработана база данных «Кинотеатр» и программное приложение для работы с ней.

Проведено исследование предметной области: описана работа кинотеатра, определены пользователи системы, сформулированы семь функций системы и восемь запросов к базе данных с указанием входных и выходных данных.

Построена инфологическая модель: выделено семь наборов объектов с атрибутами и ключами, установлено шесть связей. Связь «многие ко многим» между фильмами и жанрами разделена на две связи «один ко многим» через дополнительную сущность.

Инфологическая модель преобразована в схему реляционной базы данных из восьми таблиц, связи заданы семью внешними ключами. Предусмотрены два вычисляемых столбца и два представления для отчётности. Схема приведена к третьей нормальной форме и дополнена правилами целостности. Структура реализована средствами СУБД SQLite, таблицы заполнены контрольными данными — не менее десяти записей в каждой.

Разработано приложение на языке Python с оконным интерфейсом из пяти классов и восьми вкладок: ведение справочников, расписание, продажа и возврат билетов с автоматическим расчётом цены, поиск и отбор информации, шесть отчётов. Приложение отлажено и протестировано по девяти проверкам, включая попытки внести некорректные данные; все такие операции отклоняются с понятным сообщением, ошибок в работе не выявлено.

Цель курсового проекта достигнута, все поставленные задачи выполнены. Разработанная система может использоваться кассирами, администраторами и руководителем кинотеатра для ведения расписания, продажи билетов и получения отчётности. Направления дальнейшего развития: введение ценовых категорий мест, продажа билетов через сайт, система абонементов и бонусных баллов, а также учёт нескольких кинотеатров сети — структура базы данных позволяет внести эти изменения добавлением новых таблиц и полей без перестройки существующих связей.

Список использованной литературы

1. Карпова И. П. Базы данных: учебное пособие для вузов. — СПб.: Питер, 2023. — 240 с.
2. Кузнецов С. Д. Базы данных: учебник для студентов высших учебных заведений. — М.: Академия, 2022. — 496 с.
3. Советов Б. Я., Цехановский В. В., Чертовской В. Д. Базы данных: учебник для вузов. — 3-е изд. — М.: Юрайт, 2024. — 420 с.
4. Илюшечкин В. М. Основы использования и проектирования баз данных: учебник для вузов. — 2-е изд. — М.: Юрайт, 2023. — 213 с.
5. Лутц М. Изучаем Python. Том 1. — 5-е изд. — М.: Диалектика, 2022. — 832 с.
6. Документация по языку Python. Модуль sqlite3 [Электронный ресурс]. — Режим доступа: https://docs.python.org/3/library/sqlite3.html (дата обращения: 16.10.2026).
7. Документация по языку Python. Библиотека tkinter [Электронный ресурс]. — Режим доступа: https://docs.python.org/3/library/tkinter.html (дата обращения: 16.10.2026).
8. Документация СУБД SQLite [Электронный ресурс]. — Режим доступа: https://www.sqlite.org/docs.html (дата обращения: 16.10.2026).

Приложение 1. Исходный текст программного приложения

Ниже приведён полный текст программы `kinoteatr.py` с комментариями.

```
# -*- coding: utf-8 -*-
"""
Информационная система «Кинотеатр»
Курсовой проект по дисциплине «Базы данных»

Приложение для работы с базой данных кинотеатра.
СУБД: SQLite, файл базы данных kinoteatr.db лежит рядом с программой.
Интерфейс: библиотека tkinter (входит в стандартную поставку Python).

Запуск:  python kinoteatr.py
"""

import os
import sqlite3
import tkinter as tk
from tkinter import ttk, messagebox

PAPKA = os.path.dirname(os.path.abspath(__file__))
FAYL_BD = os.path.join(PAPKA, 'kinoteatr.db')
SKRIPT_STRUKTURY = os.path.join(PAPKA, 'kino_schema.sql')
SKRIPT_DANNYH = os.path.join(PAPKA, 'kino_dannye.sql')


# ======================================================================
#  Работа с базой данных
# ======================================================================

class BazaDannyh:
    """Класс отвечает за подключение к базе данных и выполнение запросов."""

    def __init__(self, fayl):
        self.novaya = not os.path.exists(fayl)
        self.con = sqlite3.connect(fayl)
        self.con.row_factory = sqlite3.Row
        self.con.execute('PRAGMA foreign_keys = ON')
        if self.novaya:
            self.sozdat_bazu()

    def sozdat_bazu(self):
        """Если файла базы нет, создаём таблицы и заполняем их данными."""
        for skript in (SKRIPT_STRUKTURY, SKRIPT_DANNYH):
            if os.path.exists(skript):
                with open(skript, encoding='utf-8') as f:
                    self.con.executescript(f.read())
        self.con.commit()

    def vybrat(self, sql, parametry=()):
        """Выполнить запрос на выборку и вернуть список строк."""
        return self.con.execute(sql, parametry).fetchall()

    def vypolnit(self, sql, parametry=()):
        """Выполнить запрос на изменение данных (INSERT, UPDATE, DELETE)."""
        kursor = self.con.execute(sql, parametry)
        self.con.commit()
        return kursor.lastrowid

    def zakryt(self):
        self.con.close()


# ======================================================================
#  Универсальная вкладка-справочник
# ======================================================================

class VkladkaSpravochnik(ttk.Frame):
    """Вкладка для работы с одной таблицей: просмотр, поиск, добавление,
    изменение и удаление записей."""

    def __init__(self, roditel, bd, tablica, klyuch, polya, zagolovki,
                 polya_poiska, shiriny=None):
        super().__init__(roditel, padding=8)
        self.bd = bd
```

```
        self.tablica = tablica
        self.klyuch = klyuch              # имя поля первичного ключа
        self.polya = polya                # поля, которые вводит пользователь
        self.zagolovki = zagolovki        # подписи полей на русском
        self.polya_poiska = polya_poiska  # по каким полям работает поиск
        self.shiriny = shiriny or {}
        self.vybrannyy_id = None

        self.sozdat_panel_poiska()
        self.sozdat_tablicu()
        self.sozdat_formu()
        self.obnovit()

    # ---------- создание элементов окна ----------

    def sozdat_panel_poiska(self):
        panel = ttk.Frame(self)
        panel.pack(fill='x', pady=(0, 6))
        ttk.Label(panel, text='Поиск:').pack(side='left')
        self.pole_poiska = ttk.Entry(panel, width=30)
        self.pole_poiska.pack(side='left', padx=5)
        self.pole_poiska.bind('<Return>', lambda e: self.obnovit())
        ttk.Button(panel, text='Найти', command=self.obnovit).pack(side='left')
        ttk.Button(panel, text='Показать все',
                   command=self.sbrosit_poisk).pack(side='left', padx=5)

    def sozdat_tablicu(self):
        ramka = ttk.Frame(self)
        ramka.pack(fill='both', expand=True)
        stolbcy = [self.klyuch] + self.polya
        self.tablica_vid = ttk.Treeview(ramka, columns=stolbcy,
                                        show='headings', height=12)
        for stolbec in stolbcy:
            podpis = 'Код' if stolbec == self.klyuch else self.zagolovki[stolbec]
            self.tablica_vid.heading(stolbec, text=podpis)
            self.tablica_vid.column(stolbec, width=self.shiriny.get(stolbec, 120),
                                    anchor='w')
        polosa = ttk.Scrollbar(ramka, orient='vertical',
                               command=self.tablica_vid.yview)
        self.tablica_vid.configure(yscrollcommand=polosa.set)
        self.tablica_vid.pack(side='left', fill='both', expand=True)
        polosa.pack(side='right', fill='y')
        self.tablica_vid.bind('<<TreeviewSelect>>', self.vybrat_zapis)

    def sozdat_formu(self):
        forma = ttk.LabelFrame(self, text='Запись', padding=8)
        forma.pack(fill='x', pady=8)
        self.vvod = {}
        for nomer, pole in enumerate(self.polya):
            stroka, stolbec = divmod(nomer, 3)
            ttk.Label(forma, text=self.zagolovki[pole] + ':').grid(
                row=stroka, column=stolbec * 2, sticky='e', padx=4, pady=3)
            entry = ttk.Entry(forma, width=24)
            entry.grid(row=stroka, column=stolbec * 2 + 1, padx=4, pady=3)
            self.vvod[pole] = entry

        knopki = ttk.Frame(self)
        knopki.pack(fill='x')
        ttk.Button(knopki, text='Добавить', command=self.dobavit).pack(side='left')
        ttk.Button(knopki, text='Изменить', command=self.izmenit).pack(side='left', padx=5)
        ttk.Button(knopki, text='Удалить', command=self.udalit).pack(side='left')
        ttk.Button(knopki, text='Очистить поля', command=self.ochistit).pack(side='left', padx=5)

    # ---------- работа с данными ----------

    def obnovit(self):
        """Заполнить таблицу записями с учётом строки поиска."""
        tekst = self.pole_poiska.get().strip()
        sql = f'SELECT {self.klyuch}, {", ".join(self.polya)} FROM {self.tablica}'
        parametry = ()
        if tekst:
            usloviya = ' OR '.join(f'{p} LIKE ?' for p in self.polya_poiska)
            sql += ' WHERE ' + usloviya
            parametry = tuple(f'%{tekst}%' for _ in self.polya_poiska)
        sql += f' ORDER BY {self.klyuch}'
        self.tablica_vid.delete(*self.tablica_vid.get_children())
        for stroka in self.bd.vybrat(sql, parametry):
            self.tablica_vid.insert('', 'end', values=tuple(stroka))
```

```
    def sbrosit_poisk(self):
        self.pole_poiska.delete(0, 'end')
        self.obnovit()

    def vybrat_zapis(self, sobytie=None):
        """При выборе строки в таблице её значения попадают в поля ввода."""
        vydelenie = self.tablica_vid.selection()
        if not vydelenie:
            return
        znacheniya = self.tablica_vid.item(vydelenie[0])['values']
        self.vybrannyy_id = znacheniya[0]
        for nomer, pole in enumerate(self.polya, start=1):
            self.vvod[pole].delete(0, 'end')
            self.vvod[pole].insert(0, znacheniya[nomer])

    def ochistit(self):
        self.vybrannyy_id = None
        for entry in self.vvod.values():
            entry.delete(0, 'end')
        self.tablica_vid.selection_remove(self.tablica_vid.selection())

    def sobrat_znacheniya(self):
        return [self.vvod[p].get().strip() or None for p in self.polya]

    def dobavit(self):
        znacheniya = self.sobrat_znacheniya()
        if not znacheniya[0]:
            messagebox.showwarning('Внимание', 'Заполните хотя бы первое поле.')
            return
        voprosy = ', '.join('?' for _ in self.polya)
        sql = f'INSERT INTO {self.tablica} ({", ".join(self.polya)}) VALUES ({voprosy})'
        try:
            self.bd.vypolnit(sql, tuple(znacheniya))
        except sqlite3.IntegrityError as oshibka:
            messagebox.showerror('Ошибка', f'Запись не добавлена:\n{oshibka}')
            return
        self.ochistit()
        self.obnovit()

    def izmenit(self):
        if self.vybrannyy_id is None:
            messagebox.showwarning('Внимание', 'Сначала выберите запись в таблице.')
            return
        prisvoeniya = ', '.join(f'{p} = ?' for p in self.polya)
        sql = f'UPDATE {self.tablica} SET {prisvoeniya} WHERE {self.klyuch} = ?'
        try:
            self.bd.vypolnit(sql, tuple(self.sobrat_znacheniya()) + (self.vybrannyy_id,))
        except sqlite3.IntegrityError as oshibka:
            messagebox.showerror('Ошибка', f'Запись не изменена:\n{oshibka}')
            return
        self.obnovit()

    def udalit(self):
        if self.vybrannyy_id is None:
            messagebox.showwarning('Внимание', 'Сначала выберите запись в таблице.')
            return
        if not messagebox.askyesno('Удаление', 'Удалить выбранную запись?'):
            return
        try:
            self.bd.vypolnit(f'DELETE FROM {self.tablica} WHERE {self.klyuch} = ?',
                             (self.vybrannyy_id,))
        except sqlite3.IntegrityError:
            messagebox.showerror(
                'Ошибка',
                'Запись удалить нельзя: на неё ссылаются записи других таблиц.')
            return
        self.ochistit()
        self.obnovit()


# ======================================================================
#  Вкладка «Сеансы»
# ======================================================================

class VkladkaSeansy(ttk.Frame):
    """Расписание сеансов. Фильм и зал выбираются из выпадающих списков,
    поэтому ошибиться с кодом нельзя."""

    def __init__(self, roditel, bd):
```

```
        super().__init__(roditel, padding=8)
        self.bd = bd
        self.vybrannyy_id = None

        panel = ttk.Frame(self)
        panel.pack(fill='x', pady=(0, 6))
        ttk.Label(panel, text='Дата (ГГГГ-ММ-ДД):').pack(side='left')
        self.pole_daty = ttk.Entry(panel, width=14)
        self.pole_daty.pack(side='left', padx=5)
        ttk.Button(panel, text='Показать расписание',
                   command=self.obnovit).pack(side='left')
        ttk.Button(panel, text='Все сеансы',
                   command=self.pokazat_vse).pack(side='left', padx=5)

        stolbcy = ('id', 'data', 'vremya', 'film', 'zal', 'format', 'cena', 'mest')
        podpisi = ('Код', 'Дата', 'Время', 'Фильм', 'Зал', 'Формат',
                   'Цена, руб.', 'Мест в зале')
        shiriny = (50, 90, 70, 230, 110, 80, 90, 95)
        self.tablica_vid = ttk.Treeview(self, columns=stolbcy, show='headings', height=12)
        for stolbec, podpis, shirina in zip(stolbcy, podpisi, shiriny):
            self.tablica_vid.heading(stolbec, text=podpis)
            self.tablica_vid.column(stolbec, width=shirina, anchor='w')
        self.tablica_vid.pack(fill='both', expand=True)
        self.tablica_vid.bind('<<TreeviewSelect>>', self.vybrat_zapis)

        forma = ttk.LabelFrame(self, text='Сеанс', padding=8)
        forma.pack(fill='x', pady=8)

        ttk.Label(forma, text='Фильм:').grid(row=0, column=0, sticky='e', padx=4, pady=3)
        self.spisok_filmov = ttk.Combobox(forma, width=38, state='readonly')
        self.spisok_filmov.grid(row=0, column=1, padx=4, pady=3)

        ttk.Label(forma, text='Зал:').grid(row=0, column=2, sticky='e', padx=4, pady=3)
        self.spisok_zalov = ttk.Combobox(forma, width=24, state='readonly')
        self.spisok_zalov.grid(row=0, column=3, padx=4, pady=3)

        ttk.Label(forma, text='Дата:').grid(row=1, column=0, sticky='e', padx=4, pady=3)
        self.vvod_data = ttk.Entry(forma, width=38)
        self.vvod_data.grid(row=1, column=1, padx=4, pady=3)

        ttk.Label(forma, text='Время:').grid(row=1, column=2, sticky='e', padx=4, pady=3)
        self.vvod_vremya = ttk.Entry(forma, width=24)
        self.vvod_vremya.grid(row=1, column=3, padx=4, pady=3)

        ttk.Label(forma, text='Цена, руб.:').grid(row=2, column=0, sticky='e', padx=4, pady=3)
        self.vvod_cena = ttk.Entry(forma, width=38)
        self.vvod_cena.grid(row=2, column=1, padx=4, pady=3)

        ttk.Label(forma, text='Формат:').grid(row=2, column=2, sticky='e', padx=4, pady=3)
        self.spisok_formatov = ttk.Combobox(forma, width=24, state='readonly',
                                            values=('2D', '3D', 'IMAX'))
        self.spisok_formatov.grid(row=2, column=3, padx=4, pady=3)

        knopki = ttk.Frame(self)
        knopki.pack(fill='x')
        ttk.Button(knopki, text='Добавить сеанс', command=self.dobavit).pack(side='left')
        ttk.Button(knopki, text='Изменить', command=self.izmenit).pack(side='left', padx=5)
        ttk.Button(knopki, text='Удалить', command=self.udalit).pack(side='left')
        ttk.Button(knopki, text='Очистить поля', command=self.ochistit).pack(side='left', padx=5)

        self.zagruzit_spiski()
        self.obnovit()

    def zagruzit_spiski(self):
        self.filmy = self.bd.vybrat('SELECT id_filma, nazvanie FROM film ORDER BY nazvanie')
        self.zaly = self.bd.vybrat('SELECT id_zala, nazvanie FROM zal ORDER BY nazvanie')
        self.spisok_filmov['values'] = [f'{r["id_filma"]} — {r["nazvanie"]}' for r in self.filmy]
        self.spisok_zalov['values'] = [f'{r["id_zala"]} — {r["nazvanie"]}' for r in self.zaly]

    def obnovit(self):
        data = self.pole_daty.get().strip()
        sql = ('SELECT s.id_seansa, s.data_seansa, s.vremya_nachala, f.nazvanie, '
               '       z.nazvanie, s.format_pokaza, s.bazovaya_cena, z.vsego_mest '
               'FROM seans s '
               'JOIN film f ON f.id_filma = s.id_filma '
               'JOIN zal  z ON z.id_zala  = s.id_zala ')
        parametry = ()
        if data:
            sql += 'WHERE s.data_seansa = ? '
```

```
            parametry = (data,)
        sql += 'ORDER BY s.data_seansa, s.vremya_nachala'
        self.tablica_vid.delete(*self.tablica_vid.get_children())
        for stroka in self.bd.vybrat(sql, parametry):
            self.tablica_vid.insert('', 'end', values=tuple(stroka))

    def pokazat_vse(self):
        self.pole_daty.delete(0, 'end')
        self.obnovit()

    def vybrat_zapis(self, sobytie=None):
        vydelenie = self.tablica_vid.selection()
        if not vydelenie:
            return
        znacheniya = self.tablica_vid.item(vydelenie[0])['values']
        self.vybrannyy_id = znacheniya[0]
        stroka = self.bd.vybrat('SELECT * FROM seans WHERE id_seansa = ?',
                                (self.vybrannyy_id,))[0]
        self.ustanovit_spisok(self.spisok_filmov, self.filmy, 'id_filma', stroka['id_filma'])
        self.ustanovit_spisok(self.spisok_zalov, self.zaly, 'id_zala', stroka['id_zala'])
        self.vvod_data.delete(0, 'end'); self.vvod_data.insert(0, stroka['data_seansa'])
        self.vvod_vremya.delete(0, 'end'); self.vvod_vremya.insert(0, stroka['vremya_nachala'])
        self.vvod_cena.delete(0, 'end'); self.vvod_cena.insert(0, stroka['bazovaya_cena'])
        self.spisok_formatov.set(stroka['format_pokaza'])

    @staticmethod
    def ustanovit_spisok(spisok, dannye, pole_koda, kod):
        for nomer, stroka in enumerate(dannye):
            if stroka[pole_koda] == kod:
                spisok.current(nomer)
                return

    def ochistit(self):
        self.vybrannyy_id = None
        self.spisok_filmov.set(''); self.spisok_zalov.set(''); self.spisok_formatov.set('')
        for pole in (self.vvod_data, self.vvod_vremya, self.vvod_cena):
            pole.delete(0, 'end')

    def sobrat_dannye(self):
        if not self.spisok_filmov.get() or not self.spisok_zalov.get():
            messagebox.showwarning('Внимание', 'Выберите фильм и зал.')
            return None
        try:
            cena = float(self.vvod_cena.get().replace(',', '.'))
        except ValueError:
            messagebox.showwarning('Внимание', 'Цена должна быть числом.')
            return None
        return (int(self.spisok_filmov.get().split(' — ')[0]),
                int(self.spisok_zalov.get().split(' — ')[0]),
                self.vvod_data.get().strip(),
                self.vvod_vremya.get().strip(),
                cena,
                self.spisok_formatov.get() or '2D')

    def dobavit(self):
        dannye = self.sobrat_dannye()
        if not dannye:
            return
        try:
            self.bd.vypolnit(
                'INSERT INTO seans (id_filma, id_zala, data_seansa, vremya_nachala, '
                'bazovaya_cena, format_pokaza) VALUES (?, ?, ?, ?, ?, ?)', dannye)
        except sqlite3.IntegrityError as oshibka:
            messagebox.showerror(
                'Ошибка',
                'Сеанс не добавлен. Возможно, в этом зале на это время уже назначен '
                f'другой сеанс.\n\n{oshibka}')
            return
        self.ochistit()
        self.obnovit()

    def izmenit(self):
        if self.vybrannyy_id is None:
            messagebox.showwarning('Внимание', 'Сначала выберите сеанс в таблице.')
            return
        dannye = self.sobrat_dannye()
        if not dannye:
            return
        try:
```

```
            self.bd.vypolnit(
                'UPDATE seans SET id_filma = ?, id_zala = ?, data_seansa = ?, '
                'vremya_nachala = ?, bazovaya_cena = ?, format_pokaza = ? '
                'WHERE id_seansa = ?', dannye + (self.vybrannyy_id,))
        except sqlite3.IntegrityError as oshibka:
            messagebox.showerror('Ошибка', f'Сеанс не изменён:\n{oshibka}')
            return
        self.obnovit()

    def udalit(self):
        if self.vybrannyy_id is None:
            messagebox.showwarning('Внимание', 'Сначала выберите сеанс в таблице.')
            return
        if not messagebox.askyesno('Удаление', 'Удалить выбранный сеанс?'):
            return
        try:
            self.bd.vypolnit('DELETE FROM seans WHERE id_seansa = ?', (self.vybrannyy_id,))
        except sqlite3.IntegrityError:
            messagebox.showerror('Ошибка',
                                 'Сеанс удалить нельзя: на него уже проданы билеты.')
            return
        self.ochistit()
        self.obnovit()


# ======================================================================
#  Вкладка «Продажа билетов»
# ======================================================================

class VkladkaBilety(ttk.Frame):
    """Продажа и возврат билетов. Итоговая цена считается автоматически
    (вычисляемый столбец в базе данных), занятые места проверяются."""

    def __init__(self, roditel, bd):
        super().__init__(roditel, padding=8)
        self.bd = bd
        self.vybrannyy_id = None

        forma = ttk.LabelFrame(self, text='Продажа билета', padding=8)
        forma.pack(fill='x')

        ttk.Label(forma, text='Сеанс:').grid(row=0, column=0, sticky='e', padx=4, pady=3)
        self.spisok_seansov = ttk.Combobox(forma, width=60, state='readonly')
        self.spisok_seansov.grid(row=0, column=1, columnspan=3, padx=4, pady=3, sticky='w')
        self.spisok_seansov.bind('<<ComboboxSelected>>', self.vybran_seans)

        ttk.Label(forma, text='Клиент:').grid(row=1, column=0, sticky='e', padx=4, pady=3)
        self.spisok_klientov = ttk.Combobox(forma, width=34, state='readonly')
        self.spisok_klientov.grid(row=1, column=1, padx=4, pady=3, sticky='w')
        self.spisok_klientov.bind('<<ComboboxSelected>>', self.poschitat_cenu)

        ttk.Label(forma, text='Кассир:').grid(row=1, column=2, sticky='e', padx=4, pady=3)
        self.spisok_sotrudnikov = ttk.Combobox(forma, width=26, state='readonly')
        self.spisok_sotrudnikov.grid(row=1, column=3, padx=4, pady=3, sticky='w')

        ttk.Label(forma, text='Ряд:').grid(row=2, column=0, sticky='e', padx=4, pady=3)
        self.vvod_ryad = ttk.Spinbox(forma, from_=1, to=30, width=6)
        self.vvod_ryad.grid(row=2, column=1, padx=4, pady=3, sticky='w')

        ttk.Label(forma, text='Место:').grid(row=2, column=2, sticky='e', padx=4, pady=3)
        self.vvod_mesto = ttk.Spinbox(forma, from_=1, to=30, width=6)
        self.vvod_mesto.grid(row=2, column=3, padx=4, pady=3, sticky='w')

        self.nadpis_ceny = ttk.Label(forma, text='Цена билета: —', font=('Segoe UI', 10, 'bold'))
        self.nadpis_ceny.grid(row=3, column=0, columnspan=4, sticky='w', padx=4, pady=6)

        knopki = ttk.Frame(self)
        knopki.pack(fill='x', pady=(0, 8))
        ttk.Button(knopki, text='Продать билет', command=self.prodat).pack(side='left')
        ttk.Button(knopki, text='Свободные места',
                   command=self.pokazat_svobodnye).pack(side='left', padx=5)
        ttk.Button(knopki, text='Вернуть билет', command=self.vernut).pack(side='left')
        ttk.Button(knopki, text='Удалить билет', command=self.udalit).pack(side='left', padx=5)

        stolbcy = ('id', 'seans', 'film', 'klient', 'ryad', 'mesto', 'cena', 'status')
        podpisi = ('Код', 'Дата и время', 'Фильм', 'Клиент', 'Ряд', 'Место',
                   'Итоговая цена', 'Статус')
        shiriny = (50, 120, 200, 160, 50, 60, 110, 110)
        self.tablica_vid = ttk.Treeview(self, columns=stolbcy, show='headings', height=12)
```

```
        for stolbec, podpis, shirina in zip(stolbcy, podpisi, shiriny):
            self.tablica_vid.heading(stolbec, text=podpis)
            self.tablica_vid.column(stolbec, width=shirina, anchor='w')
        self.tablica_vid.pack(fill='both', expand=True)
        self.tablica_vid.bind('<<TreeviewSelect>>', self.vybrat_zapis)

        self.zagruzit_spiski()
        self.obnovit()

    def zagruzit_spiski(self):
        self.seansy = self.bd.vybrat(
            'SELECT s.id_seansa, s.data_seansa, s.vremya_nachala, f.nazvanie AS film, '
            '       z.nazvanie AS zal, s.bazovaya_cena, z.kolichestvo_ryadov, z.mest_v_ryadu '
            'FROM seans s JOIN film f ON f.id_filma = s.id_filma '
            'JOIN zal z ON z.id_zala = s.id_zala '
            'ORDER BY s.data_seansa, s.vremya_nachala')
        self.klienty = self.bd.vybrat(
            'SELECT id_klienta, familiya, imya, skidka_procent FROM klient ORDER BY familiya')
        self.sotrudniki = self.bd.vybrat(
            "SELECT id_sotrudnika, familiya, imya FROM sotrudnik "
            "WHERE dolzhnost IN ('кассир','администратор') ORDER BY familiya")

        self.spisok_seansov['values'] = [
            f'{r["id_seansa"]} — {r["data_seansa"]} {r["vremya_nachala"]} — '
            f'{r["film"]} ({r["zal"]}, {r["bazovaya_cena"]:.0f} руб.)' for r in self.seansy]
        self.spisok_klientov['values'] = ['без карты'] + [
            f'{r["id_klienta"]} — {r["familiya"]} {r["imya"]} (скидка {r["skidka_procent"]:.0f}%)'
            for r in self.klienty]
        self.spisok_sotrudnikov['values'] = [
            f'{r["id_sotrudnika"]} — {r["familiya"]} {r["imya"]}' for r in self.sotrudniki]

    # ---------- вспомогательные действия ----------

    def tekushchiy_seans(self):
        if not self.spisok_seansov.get():
            return None
        kod = int(self.spisok_seansov.get().split(' — ')[0])
        for stroka in self.seansy:
            if stroka['id_seansa'] == kod:
                return stroka
        return None

    def tekushchaya_skidka(self):
        vybor = self.spisok_klientov.get()
        if not vybor or vybor == 'без карты':
            return 0.0
        kod = int(vybor.split(' — ')[0])
        for stroka in self.klienty:
            if stroka['id_klienta'] == kod:
                return float(stroka['skidka_procent'])
        return 0.0

    def vybran_seans(self, sobytie=None):
        """При выборе сеанса ограничиваем номера рядов и мест размерами зала."""
        seans = self.tekushchiy_seans()
        if seans:
            self.vvod_ryad.configure(to=seans['kolichestvo_ryadov'])
            self.vvod_mesto.configure(to=seans['mest_v_ryadu'])
        self.poschitat_cenu()

    def poschitat_cenu(self, sobytie=None):
        seans = self.tekushchiy_seans()
        if not seans:
            self.nadpis_ceny.configure(text='Цена билета: —')
            return
        skidka = self.tekushchaya_skidka()
        itog = round(seans['bazovaya_cena'] * (1 - skidka / 100), 2)
        self.nadpis_ceny.configure(
            text=f'Цена билета: {seans["bazovaya_cena"]:.2f} руб. − скидка {skidka:.0f}% '
                 f'= {itog:.2f} руб.')

    def obnovit(self):
        sql = ('SELECT b.id_bileta, s.data_seansa || " " || s.vremya_nachala, f.nazvanie, '
               '       IFNULL(k.familiya || " " || k.imya, "без карты"), '
               '       b.ryad, b.mesto, b.itogovaya_cena, b.status '
               'FROM bilet b '
               'JOIN seans s ON s.id_seansa = b.id_seansa '
               'JOIN film f ON f.id_filma = s.id_filma '
               'LEFT JOIN klient k ON k.id_klienta = b.id_klienta '
```

```
            'ORDER BY b.id_bileta DESC')
        self.tablica_vid.delete(*self.tablica_vid.get_children())
        for stroka in self.bd.vybrat(sql):
            self.tablica_vid.insert('', 'end', values=tuple(stroka))

    def vybrat_zapis(self, sobytie=None):
        vydelenie = self.tablica_vid.selection()
        if vydelenie:
            self.vybrannyy_id = self.tablica_vid.item(vydelenie[0])['values'][0]

    # ---------- основные операции ----------

    def prodat(self):
        seans = self.tekushchiy_seans()
        if not seans:
            messagebox.showwarning('Внимание', 'Выберите сеанс.')
            return
        if not self.spisok_sotrudnikov.get():
            messagebox.showwarning('Внимание', 'Выберите кассира.')
            return
        try:
            ryad = int(self.vvod_ryad.get())
            mesto = int(self.vvod_mesto.get())
        except ValueError:
            messagebox.showwarning('Внимание', 'Номер ряда и места — целые числа.')
            return
        if ryad > seans['kolichestvo_ryadov'] or mesto > seans['mest_v_ryadu']:
            messagebox.showwarning(
                'Внимание',
                f'В зале «{seans["zal"]}» {seans["kolichestvo_ryadov"]} рядов '
                f'по {seans["mest_v_ryadu"]} мест.')
            return

        vybor_klienta = self.spisok_klientov.get()
        kod_klienta = (None if not vybor_klienta or vybor_klienta == 'без карты'
                       else int(vybor_klienta.split(' — ')[0]))
        kod_sotrudnika = int(self.spisok_sotrudnikov.get().split(' — ')[0])
        skidka = self.tekushchaya_skidka()

        try:
            self.bd.vypolnit(
                'INSERT INTO bilet (id_seansa, id_klienta, id_sotrudnika, ryad, mesto, '
                'data_prodazhi, cena_bez_skidki, skidka_procent, status) '
                "VALUES (?, ?, ?, ?, ?, date('now'), ?, ?, 'продан')",
                (seans['id_seansa'], kod_klienta, kod_sotrudnika, ryad, mesto,
                 seans['bazovaya_cena'], skidka))
        except sqlite3.IntegrityError:
            messagebox.showerror('Ошибка',
                                 f'Место {mesto} в ряду {ryad} на этот сеанс уже продано.')
            return
        itog = round(seans['bazovaya_cena'] * (1 - skidka / 100), 2)
        messagebox.showinfo('Билет продан',
                            f'Фильм: {seans["film"]}\nЗал: {seans["zal"]}\n'
                            f'Ряд {ryad}, место {mesto}\nК оплате: {itog:.2f} руб.')
        self.obnovit()

    def pokazat_svobodnye(self):
        seans = self.tekushchiy_seans()
        if not seans:
            messagebox.showwarning('Внимание', 'Выберите сеанс.')
            return
        zanyato = {(r['ryad'], r['mesto']) for r in self.bd.vybrat(
            "SELECT ryad, mesto FROM bilet WHERE id_seansa = ? AND status <> 'возвращён'",
            (seans['id_seansa'],))}
        vsego = seans['kolichestvo_ryadov'] * seans['mest_v_ryadu']
        svobodno = vsego - len(zanyato)
        spisok_zanyatyh = ', '.join(f'ряд {r}, место {m}' for r, m in sorted(zanyato)) or 'нет'
        messagebox.showinfo(
            'Свободные места',
            f'Фильм: {seans["film"]}\nЗал: {seans["zal"]}\n'
            f'Всего мест: {vsego}\nСвободно: {svobodno}\n\nЗанятые места:\n{spisok_zanyatyh}')

    def vernut(self):
        if self.vybrannyy_id is None:
            messagebox.showwarning('Внимание', 'Выберите билет в таблице.')
            return
        if not messagebox.askyesno('Возврат', 'Оформить возврат выбранного билета?'):
            return
        self.bd.vypolnit("UPDATE bilet SET status = 'возвращён' WHERE id_bileta = ?",
```

```
                        (self.vybrannyy_id,))
        self.obnovit()

    def udalit(self):
        if self.vybrannyy_id is None:
            messagebox.showwarning('Внимание', 'Выберите билет в таблице.')
            return
        if not messagebox.askyesno('Удаление', 'Удалить запись о билете?'):
            return
        self.bd.vypolnit('DELETE FROM bilet WHERE id_bileta = ?', (self.vybrannyy_id,))
        self.vybrannyy_id = None
        self.obnovit()


# ======================================================================
#  Вкладка «Отчёты»
# ======================================================================

class VkladkaOtchety(ttk.Frame):
    """Отчёты по работе кинотеатра. Каждый отчёт — это запрос к базе данных."""

    OTCHETY = (
        'Расписание сеансов на дату',
        'Заполненность залов и выручка по сеансам',
        'Выручка за период',
        'Рейтинг фильмов по числу проданных билетов',
        'Продажи по кассирам',
        'Фильмы по жанру',
    )

    def __init__(self, roditel, bd):
        super().__init__(roditel, padding=8)
        self.bd = bd

        panel = ttk.Frame(self)
        panel.pack(fill='x', pady=(0, 6))
        ttk.Label(panel, text='Отчёт:').pack(side='left')
        self.spisok_otchetov = ttk.Combobox(panel, width=46, state='readonly',
                                            values=self.OTCHETY)
        self.spisok_otchetov.current(0)
        self.spisok_otchetov.pack(side='left', padx=5)
        self.spisok_otchetov.bind('<<ComboboxSelected>>', self.podstavit_parametry)

        ttk.Label(panel, text='Параметр 1:').pack(side='left', padx=(10, 2))
        self.parametr1 = ttk.Entry(panel, width=14)
        self.parametr1.pack(side='left')

        ttk.Label(panel, text='Параметр 2:').pack(side='left', padx=(10, 2))
        self.parametr2 = ttk.Entry(panel, width=14)
        self.parametr2.pack(side='left')

        ttk.Button(panel, text='Построить отчёт',
                   command=self.postroit).pack(side='left', padx=8)

        self.podskazka = ttk.Label(self, foreground='#444444', text='')
        self.podskazka.pack(fill='x', pady=(0, 6))

        self.tablica_vid = ttk.Treeview(self, show='headings', height=16)
        self.tablica_vid.pack(fill='both', expand=True)
        self.itog = ttk.Label(self, text='', font=('Segoe UI', 10, 'bold'))
        self.itog.pack(fill='x', pady=6)

        self.podstavit_parametry()
        self.postroit()

    def podstavit_parametry(self, sobytie=None):
        """Подставляет в поля параметров подходящие значения для выбранного отчёта,
        чтобы не вводить даты вручную."""
        nazvanie = self.spisok_otchetov.get()
        self.parametr1.delete(0, 'end')
        self.parametr2.delete(0, 'end')

        if nazvanie == self.OTCHETY[0]:          # расписание на дату
            data = self.bd.vybrat('SELECT MIN(data_seansa) FROM seans')[0][0]
            self.parametr1.insert(0, data or '')
            podskazka = 'Параметр 1 — дата сеансов в формате ГГГГ-ММ-ДД.'
        elif nazvanie == self.OTCHETY[2]:        # выручка за период
            granicy = self.bd.vybrat(
                'SELECT MIN(data_prodazhi), MAX(data_prodazhi) FROM bilet')[0]
```

```
            self.parametr1.insert(0, granicy[0] or '')
            self.parametr2.insert(0, granicy[1] or '')
            podskazka = 'Параметр 1 — начало периода, параметр 2 — конец периода (по дате продажи).'
        elif nazvanie == self.OTCHETY[5]:        # фильмы по жанру
            zhanr = self.bd.vybrat('SELECT nazvanie FROM zhanr ORDER BY id_zhanra')[0][0]
            self.parametr1.insert(0, zhanr)
            podskazka = 'Параметр 1 — название жанра или его часть.'
        else:
            podskazka = 'Этот отчёт строится по всем данным, параметры не нужны.'
        self.podskazka.configure(text=podskazka)

    def postroit(self):
        nazvanie = self.spisok_otchetov.get()
        p1 = self.parametr1.get().strip()
        p2 = self.parametr2.get().strip()

        if nazvanie == self.OTCHETY[0]:
            podpisi = ('Время', 'Фильм', 'Длительность, мин', 'Зал', 'Формат', 'Цена, руб.')
            sql = ('SELECT vremya_nachala, film, dlitelnost_min, zal, format_pokaza, '
                   'bazovaya_cena FROM v_raspisanie WHERE data_seansa = ? '
                   'ORDER BY vremya_nachala')
            dannye = self.bd.vybrat(sql, (p1,))
            itog = f'Сеансов на {p1}: {len(dannye)}'

        elif nazvanie == self.OTCHETY[1]:
            podpisi = ('Дата', 'Время', 'Фильм', 'Зал', 'Мест', 'Продано',
                       'Выручка, руб.', 'Заполненность, %')
            sql = ('SELECT data_seansa, vremya_nachala, film, zal, vsego_mest, '
                   'prodano_biletov, vyruchka, zapolnennost_procent '
                   'FROM v_seans_svodka ORDER BY data_seansa, vremya_nachala')
            dannye = self.bd.vybrat(sql)
            itog = f'Всего сеансов: {len(dannye)}'

        elif nazvanie == self.OTCHETY[2]:
            podpisi = ('Дата продажи', 'Продано билетов', 'Выручка, руб.')
            sql = ('SELECT data_prodazhi, COUNT(*), ROUND(SUM(itogovaya_cena), 2) '
                   "FROM bilet WHERE status = 'продан' AND data_prodazhi BETWEEN ? AND ? "
                   'GROUP BY data_prodazhi ORDER BY data_prodazhi')
            dannye = self.bd.vybrat(sql, (p1, p2))
            vsego = sum(stroka[2] for stroka in dannye) if dannye else 0
            itog = f'Выручка за период с {p1} по {p2}: {vsego:.2f} руб.'

        elif nazvanie == self.OTCHETY[3]:
            podpisi = ('Фильм', 'Режиссёр', 'Продано билетов', 'Выручка, руб.')
            sql = ('SELECT f.nazvanie, f.rezhisser, COUNT(b.id_bileta), '
                   'ROUND(IFNULL(SUM(b.itogovaya_cena), 0), 2) '
                   'FROM film f '
                   'LEFT JOIN seans s ON s.id_filma = f.id_filma '
                   "LEFT JOIN bilet b ON b.id_seansa = s.id_seansa AND b.status = 'продан' "
                   'GROUP BY f.id_filma, f.nazvanie, f.rezhisser '
                   'ORDER BY COUNT(b.id_bileta) DESC')
            dannye = self.bd.vybrat(sql)
            itog = f'Всего фильмов в прокате: {len(dannye)}'

        elif nazvanie == self.OTCHETY[4]:
            podpisi = ('Сотрудник', 'Должность', 'Оформлено билетов', 'Сумма продаж, руб.')
            sql = ('SELECT s.familiya || " " || s.imya, s.dolzhnost, COUNT(b.id_bileta), '
                   'ROUND(IFNULL(SUM(b.itogovaya_cena), 0), 2) '
                   'FROM sotrudnik s '
                   "LEFT JOIN bilet b ON b.id_sotrudnika = s.id_sotrudnika "
                   "AND b.status = 'продан' "
                   'GROUP BY s.id_sotrudnika, s.familiya, s.imya, s.dolzhnost '
                   'HAVING COUNT(b.id_bileta) > 0 '
                   'ORDER BY COUNT(b.id_bileta) DESC')
            dannye = self.bd.vybrat(sql)
            itog = f'Кассиров с продажами: {len(dannye)}'

        else:
            podpisi = ('Фильм', 'Режиссёр', 'Год', 'Длительность, мин', 'Рейтинг')
            sql = ('SELECT f.nazvanie, f.rezhisser, f.god_vypuska, f.dlitelnost_min, '
                   'f.vozrastnoy_reyting '
                   'FROM film f '
                   'JOIN film_zhanr fz ON fz.id_filma = f.id_filma '
                   'JOIN zhanr z ON z.id_zhanra = fz.id_zhanra '
                   'WHERE z.nazvanie LIKE ? ORDER BY f.nazvanie')
            dannye = self.bd.vybrat(sql, (f'%{p1}%',))
            itog = f'Найдено фильмов: {len(dannye)}'

        self.tablica_vid.delete(*self.tablica_vid.get_children())
```

```
        self.tablica_vid['columns'] = podpisi
        for podpis in podpisi:
            self.tablica_vid.heading(podpis, text=podpis)
            shirina = 260 if 'Фильм' in podpis else 140
            self.tablica_vid.column(podpis, width=shirina, anchor='w')
        for stroka in dannye:
            self.tablica_vid.insert('', 'end', values=tuple(stroka))
        self.itog.configure(text=itog)


# ======================================================================
#  Главное окно
# ======================================================================

class GlavnoeOkno(tk.Tk):

    def __init__(self):
        super().__init__()
        self.title('Информационная система «Кинотеатр»')
        self.geometry('1150x720')
        self.bd = BazaDannyh(FAYL_BD)

        zagolovok = ttk.Label(self, text='Информационная система «Кинотеатр»',
                              font=('Segoe UI', 14, 'bold'), padding=8)
        zagolovok.pack()

        vkladki = ttk.Notebook(self)
        vkladki.pack(fill='both', expand=True, padx=8, pady=8)

        vkladki.add(VkladkaSeansy(vkladki, self.bd), text='Сеансы')
        vkladki.add(VkladkaBilety(vkladki, self.bd), text='Продажа билетов')

        vkladki.add(VkladkaSpravochnik(
            vkladki, self.bd, 'film', 'id_filma',
            ['nazvanie', 'rezhisser', 'strana', 'god_vypuska', 'dlitelnost_min',
             'vozrastnoy_reyting'],
            {'nazvanie': 'Название', 'rezhisser': 'Режиссёр', 'strana': 'Страна',
             'god_vypuska': 'Год', 'dlitelnost_min': 'Длительность, мин',
             'vozrastnoy_reyting': 'Рейтинг'},
            ['nazvanie', 'rezhisser', 'strana'],
            {'id_filma': 50, 'nazvanie': 260, 'rezhisser': 170, 'god_vypuska': 60,
             'dlitelnost_min': 130, 'vozrastnoy_reyting': 80}), text='Фильмы')

        vkladki.add(VkladkaSpravochnik(
            vkladki, self.bd, 'zal', 'id_zala',
            ['nazvanie', 'kolichestvo_ryadov', 'mest_v_ryadu', 'tip_zala'],
            {'nazvanie': 'Название зала', 'kolichestvo_ryadov': 'Рядов',
             'mest_v_ryadu': 'Мест в ряду', 'tip_zala': 'Тип зала'},
            ['nazvanie', 'tip_zala'],
            {'id_zala': 50, 'nazvanie': 180}), text='Залы')

        vkladki.add(VkladkaSpravochnik(
            vkladki, self.bd, 'klient', 'id_klienta',
            ['familiya', 'imya', 'telefon', 'email', 'data_registracii', 'skidka_procent'],
            {'familiya': 'Фамилия', 'imya': 'Имя', 'telefon': 'Телефон',
             'email': 'E-mail', 'data_registracii': 'Дата регистрации',
             'skidka_procent': 'Скидка, %'},
            ['familiya', 'imya', 'telefon'],
            {'id_klienta': 50, 'email': 170, 'data_registracii': 130}), text='Клиенты')

        vkladki.add(VkladkaSpravochnik(
            vkladki, self.bd, 'sotrudnik', 'id_sotrudnika',
            ['familiya', 'imya', 'dolzhnost', 'telefon', 'data_priema'],
            {'familiya': 'Фамилия', 'imya': 'Имя', 'dolzhnost': 'Должность',
             'telefon': 'Телефон', 'data_priema': 'Дата приёма'},
            ['familiya', 'dolzhnost'],
            {'id_sotrudnika': 50, 'dolzhnost': 170, 'data_priema': 120}), text='Сотрудники')

        vkladki.add(VkladkaSpravochnik(
            vkladki, self.bd, 'zhanr', 'id_zhanra', ['nazvanie'],
            {'nazvanie': 'Название жанра'}, ['nazvanie'],
            {'id_zhanra': 60, 'nazvanie': 240}), text='Жанры')

        vkladki.add(VkladkaOtchety(vkladki, self.bd), text='Отчёты')

        self.protocol('WM_DELETE_WINDOW', self.zakryt)

    def zakryt(self):
        self.bd.zakryt()
```

```
        self.destroy()


if __name__ == '__main__':
    GlavnoeOkno().mainloop()
```